

TEKNİK ANALİZ DOKÜMANI

Finans Portalı

Belge: Teknik Analiz Dokümanı · Sürüm: 2.0

Hedef okuyucu: Yazılım mimarları, geliştiriciler ve teknik değerlendirciler

Mimari: React SPA · Spring Boot · PostgreSQL · Keycloak · Kafka · OpenTelemetry · Docker/Kubernetes

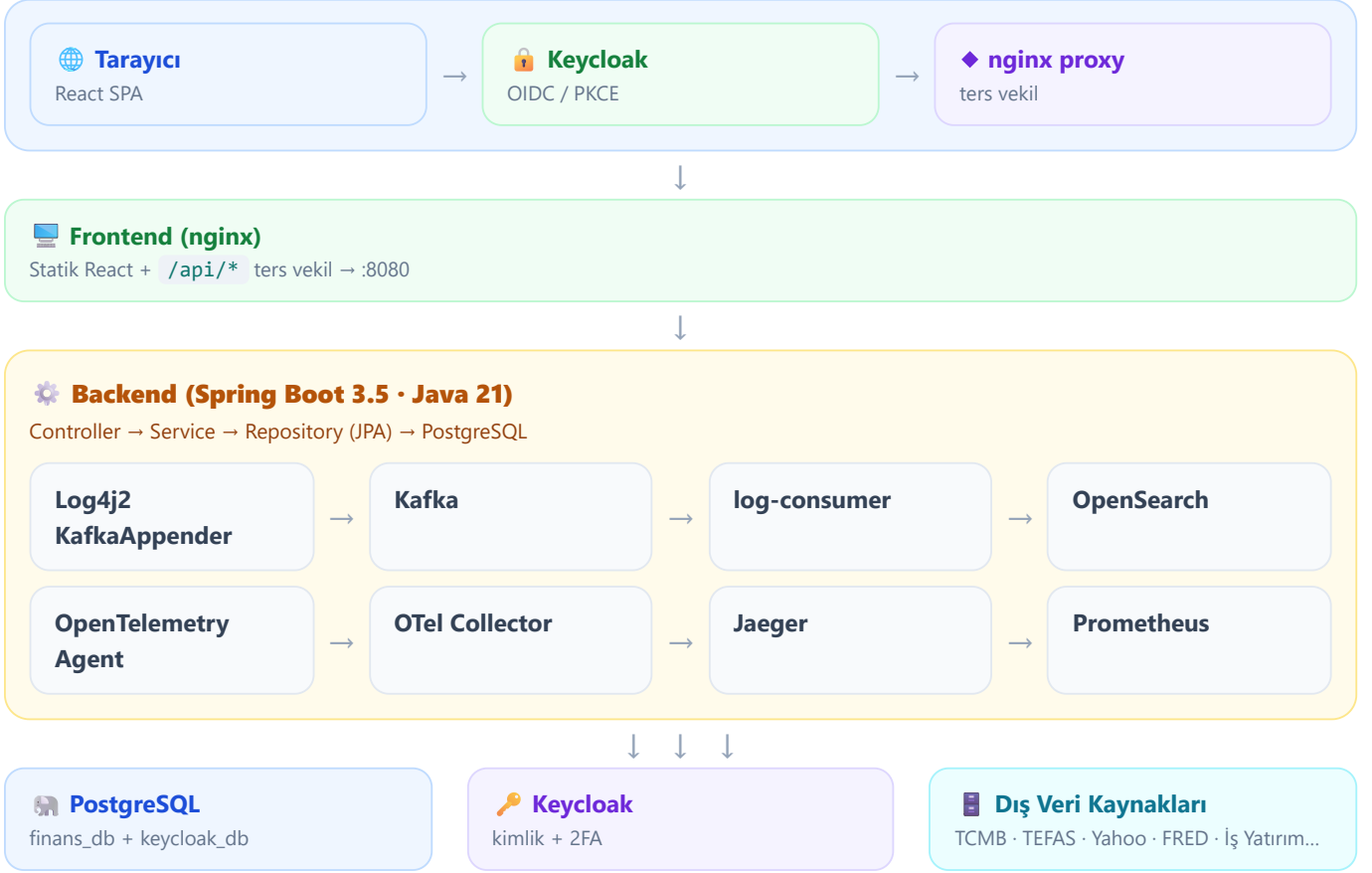
İş ve kullanıcı değeri için ayrı "İş ve Ürün Analizi Dokümanı"na bakınız.

• İindekiler

1. Mimari Genel Bakış
2. Teknoloji Yığını
3. Sistem Diyagramları
4. Veri Akışları
5. Güvenlik Mimarisi
6. Performans ve Önbellekleme
7. Hata Yönetimi
8. Test ve Kod Kalitesi
9. Dağıtım (Deployment) ve CI/CD
10. Veri Modeli
11. API Tasarımı
12. Gözlemlenebilirlik (Observability)
13. Frontend Mimarisi
14. Bilinen Sınırlamalar
15. Geliştirici Onboarding

1 Mimari Genel Bakış

Finans Portalı, **3 katmanlı (SPA → REST API → veritabanı)** bir uygulama olup yanında tam bir **gözlemlenebilirlik (logs + metrics + traces)** ve **kimlik yönetimi** yığını barındırır.



Katmanlar net biçimde ayrılmıştır: **Controller** (HTTP), **Service** (iş mantığı), **Repository** (veri erişimi), **Entity/DTO** (model). Dış kaynaklara erişim ayrı **client** paketinde izole edilmiştir. Tüm yazma uçları kimlik doğrulamalıdır; okuma uçlarının bir kısmı kamuya açıktır.

2 Teknoloji Yığını

2.1 Frontend

Bileşen	Sürüm	Rol
React	19.2	UI framework
Vite	7.x (SWC)	Build aracı + dev sunucu
React Router DOM	7.x	İstemci tarafı yönlendirme
Zustand	5.x	Hafif global durum yönetimi
Axios	1.x	HTTP istemcisi
Keycloak JS	26.x	OAuth2/OIDC PKCE akışı
Recharts	3.x	Pasta/bar grafikleri (dağılım, enflasyon)
Lightweight Charts	5.x	Alan/fiyat & tahvil getiri grafikleri (canvas)
KLineCharts	9.8	Detaylı mum grafiği + çizim araçları + indikatörler
react-window	2.x	Uzun listelerde sanal kaydırma
SheetJS (xlsx)	0.20	Excel ile portföy içe aktarma/örnek dosya
react-hot-toast	2.x	Bildirim (toast) sistemi
Tailwind + PostCSS	—	Yardımcı sınıflar (inline stil ağırlıklı)

2.2 Backend

Bileşen	Sürüm	Rol
Java	21 (LTS)	Çalışma zamanı
Spring Boot	3.5.x	Uygulama çerçevesi
Spring Web MVC	6.x	REST controller'lar
Spring WebFlux (WebClient)	6.x	Reaktif dış servis çağrıları
Spring Data JPA / Hibernate	3.x / 6.x	ORM katmanı
Spring Security (OAuth2 RS)	6.x	JWT doğrulama (Resource Server)
Spring Cache + Caffeine	—	In-memory ön bellek
Spring Kafka	3.x	Log tüketici (log-consumer)
Flyway	10.x	Şema göçü (V1–V26)
Log4j2 + JSON layout	2.24	Loglama + Kafka appender
Micrometer + OTel köprüsü	1.x	Metrik + iz (trace)
Apache POI	—	Excel okuma/yazma (sunucu tarafı)
Spring Mail	—	E-posta (alarm bildirimleri)
SpringDoc OpenAPI	2.7	Swagger UI / OpenAPI 3
Lombok	1.18	Boilerplate azaltma

2.3 Altyapı & Çevre Bileşenleri

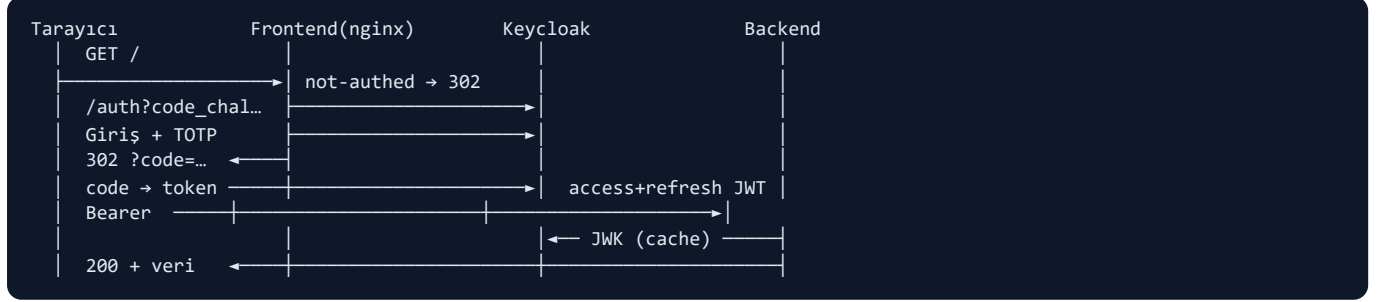
Kategori	Teknoloji
Veritabanı	PostgreSQL 15 (üretim), H2 (test, in-memory)
Kimlik sağlayıcı	Keycloak 26 (OIDC + 2FA TOTP), opsiyonel OpenLDAP federasyonu
Mesajlaşma	Apache Kafka 7.5 (Confluent) + Zookeeper
İzleme / Loglar	OpenSearch 2.11 + Dashboards, log-consumer (Spring Boot)
Metrik	Prometheus + Grafana (4 dashboard)
İz (Trace)	OpenTelemetry Collector + Jaeger
Çeviri	LibreTranslate (self-hosted, TR↔EN)
Yapay zeka (LLM)	OpenAI-uyumlu istemci (yapılandırılabilir taban URL/model; örn. Groq 11ama-3.3-70b veya OpenAI gpt-4o-mini)
Headless tarayıcı	Playwright servisi (Cloudflare korumalı sayfa içerik çekimi)
E-posta (dev/prod)	Mailpit (dev SMTP) / SMTP (prod)
Konteyner / Orkestrasyon	Docker + Docker Compose; Kubernetes (Kustomize) + GKE
CI/CD	GitHub Actions (CI + GKE'ye CD), SonarCloud

3 Sistem Diyagramları

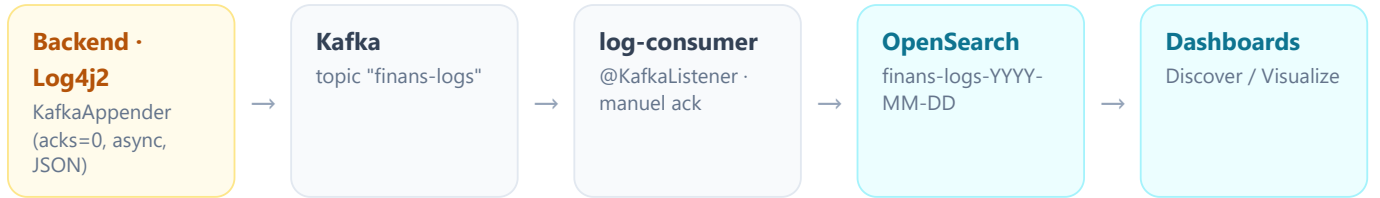
3.1 Dağıtım yığını (Docker Compose)

```
postgres · keycloak (+ keycloak-bootstrap) · backend · frontend(nginx)
kafka + zookeeper · log-consumer · opensearch + dashboards
prometheus · grafana · jaeger · otel-collector
libretranslate · playwright-service · mailpit · openldap (+ phpLdapAdmin)
```

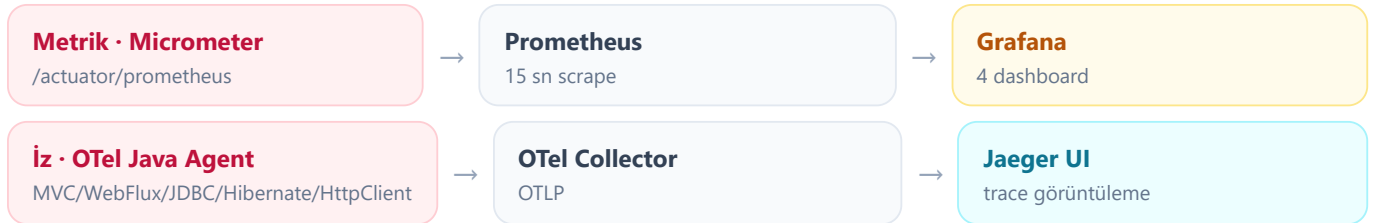
3.2 Kimlik doğrulama akışı (OIDC / PKCE)



3.3 Log hattı (Log4j2 → Kafka → OpenSearch)



3.4 Metrik & İz hatları

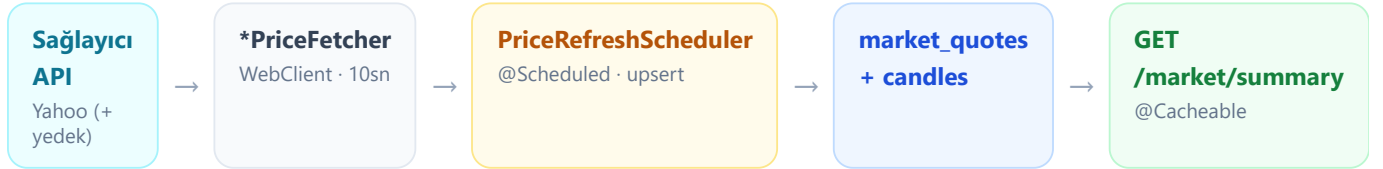


4 Veri Akışları

4.1 Piyasa fiyatı (çok sağlayıcı)

Fiyat verisi, sağlayıcı-bağımsız bir katmanla çekilir: `YahooPriceFetcher` (birincil), `TwelveDataFetcher` ve `FinnhubPriceFetcher` (alternatif/yedek). `PriceRefreshScheduler` üç tetikleyiciyle çalışır:

- **Açılışta** (startup) — tüm enstrümanlar için kota + 1 yıllık mum geçmişi.
- **Günlük 18:00 UTC** — tüm enstrümanlar (kapanış).
- **15 dakikada bir (intraday)** — yalnızca gecikmesiz (`delayed=false`) enstrümanlar; BIST hisseleri gibi gecikmeli olanlar gün-sonu yenilenir.



4.2 Tahvil getiri verisi ve geçmiş (EVDS3, YTM)

`EvdsBondYieldFetcher`, TCMB EVDS3'ten devlet tahvili fiyat + kupon serilerini çeker ve **nakit akışlarını iskontolayarak vadeye kadar getiri (YTM)** hesaplar (yarı yıllık kupon takvimi yeniden kurulur, birikmiş faiz ayrıştırılarak temiz fiyat bulunur, bisection ile yıllık oran çözülür).

Getiri geçmişi (YTM zaman serisi)

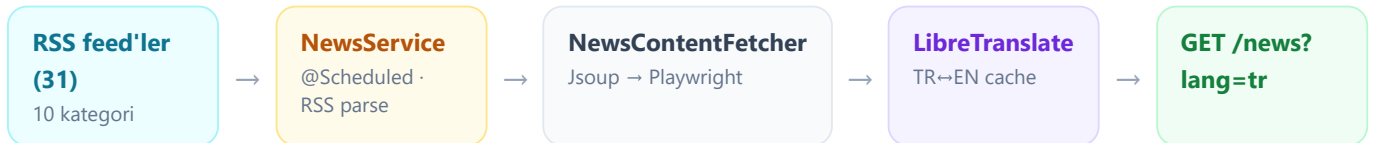
EVDS3'ten **yapılandırılabilir bir günlük pencere** (`app.bonds.tcmb.history-days`, varsayılan 120 gün) çekilir ve **her işlem günü için bir YTM** hesaplanıp saklanır; böylece her enstrümanın getiri grafiği zaman içindeki seyri gösterir. Yenileme servisi kayıtları ISIN'e göre gruplar, enstrümanı bir kez upsert eder ve tarih aralığını tek sorguyla ön-getirir (N+1 önlenir), ardından günlük kotaları yazar. Yenileme **2 saatte bir + açılışta** çalışır.

Mevduat faizleri `TcmbDepositRateFetcher` ile ayrıca çekilir. Sağlayıcı soyutlaması `BondDataProvider` (TCMB birincil, Demo yedek) ile yapılır.

4.3 Enflasyon (TR + ABD)



4.4 Haberler ve otomatik çeviri



4.5 Analiz tablosu ve reel getiri

`InstrumentAnalysisService`; piyasa özetini, döviz kurlarını, fonları ve enflasyonu birleştirir. Her satır için haftalık/aylık/yıllık değişim mum geçmişinden hesaplanır; `RiskProfileService` risk sınıfı, `TechnicalAnalysisService` kısa/uzun vade sinyali üretir. **Reel getiri** para-birimi duyarlıdır: TRY enstrümanlar TR TÜFE'sine, USD enstrümanlar ABD CPI'sine karşı $(1+nominal)/(1+enflasyon) - 1$ ile hesaplanır.

4.6 Yapay zeka sohbet danışmanı

```
POST /api/v1/analysis/chat (JWT) → AiAnalysisService.generateReply
├ Bütçe niyeti (regex "5000 TL") → 3 senaryo (Güvenli/Dengeli/Riskli)
├ Sembol niyeti (regex ticker) → DB'den güncel veri + sinyaller
├ LLM etkinse → finans-only sistem promptu + piyasa bağlamı
│   → LlmClient.complete() → /chat/completions
├ Anahtar-kelime fallback / yardım yanıtı
→ finalize(): "yatırım tavsiyesi değildir" + zaman damgası
```

Güvenlik kuralları

LLM yalnızca finans kapsamında yanıt verir; garanti/komut vermez. Servis devre dışıyken kural tabanlı yanıtlar devreye girer (dayanıklılık).

4.7 VİOP & Tahvil simülasyon işlemleri

Vadeli işlemler ve tahvil/bono, hisse gibi "adet × maliyet" mantığıyla değil, kendi ekonomileriyle modellenir (ayrı

`ViopTradeController` / `BondTradeController` ve pozisyon/işlem tabloları): VİOP'ta **long/short yön, kontrat büyüklüğü, teminat, net pozisyon ve kısmi kapatma**; tahvilde **nominal, temiz/kirli fiyat, birikmiş faiz, ağırlıklı ortalama maliyet, kupon ve itfa**. Tüm hesaplar sunucuda yapılır; her ekranda "gerçek emir değildir, simülasyondur" uyarısı bulunur. Mevcut hisse/kripto/fon portföyü bu değişiklikten etkilenmez.

5 Güvenlik Mimarisi

5.1 Kimlik doğrulama

- **Sağlayıcı:** Keycloak realm `finans`.
- **Akış:** Authorization Code + PKCE (frontend = public client).
- **Backend:** OAuth2 Resource Server; RS256 imzalı JWT, JWK Set URI ile doğrulanır. Erişim token'ı kısa ömürlü (~5 dk) + refresh token.

5.2 İki adımlı doğrulama (2FA / TOTP)

HMAC-SHA1, 6 hane, 30 sn periyot. İlk girişte QR kod (Google/Microsoft Authenticator, Authy uyumlu). Realm düzeyinde `CONFIGURE_TOTP` varsayılan zorunlu eylem olarak ayarlanır (keycloak-bootstrap).

5.3 Yetkilendirme

```
/api/v1/admin/**          → hasRole("ADMIN")
GET /api/v1/market/**      → permitAll (public read)
GET /api/v1/news/**        → permitAll
/api/v1/portfolio/**       → authenticated
/api/v1/analysis/**        → authenticated (AI sohbet dahil)
/api/v1/alerts|notifications|users/me/** → authenticated
anyRequest()              → authenticated
```

JWT'deki `realm_access.roles` claim'i özel bir `JwtAuthenticationConverter` ile Spring rollerine (`ROLE_user` , `ROLE_admin`) çevrilir.

5.4 CORS / CSRF / Loglama güvenliği

- **CORS:** Yalnızca yapılandırılan frontend origin'ine izin (env ile).
- **CSRF:** Stateless REST olduğundan kapalı.
- **Güvenlik başlıkları:** Spring Security varsayılanları (HSTS, X-Frame-Options, X-Content-Type-Options) + nginx başlıkları.
- **Log injection:** kullanıcı girdisi `LogSanitizer` ile temizlenir; iz kayıtlarında `Authorization` başlığı hariç tutulur; sırlar `.env` /secret ile yönetilir.

6 Performans ve Önbellekleme

6.1 Caffeine in-memory cache

Sık çağrılan okuma uçları kısa TTL'li (≈ 30 sn) adlandırılmış cache'lerle hızlandırılır: `market-summary`, `exchange-rates`, `investment-funds`, `inflation-*`, `bondsList` vb. Yenileme sonrası `@CacheEvict` ile geçersiz kılınır.

6.2 Zamanlayıcı havuzu

Zamanlanmış görevler 4 iş parçacıklı havuzda paralel çalışır (uzun süren haber çekimi diğer yenilemeleri bloke etmez).

6.3 Log üreticisi (Kafka)

`acks=0` (fire-and-forget), `linger.ms=1000` (batch), `max.block.ms=0` (broker yoksa anında düşür) — log yolu hiçbir zaman istek yolunu bloke etmez.

6.4 nginx dinamik upstream

Dinamik upstream çözümleme

nginx, backend host'unu Docker DNS ile **her istekte yeniden çözer** (`resolver 127.0.0.11` + değişkenli `proxy_pass`). Böylece backend konteyneri yeniden başlatılıp IP değiştiğinde frontend'i yeniden başlatmaya gerek kalmadan ~ 30 sn içinde otomatik toparlanır; stale IP kaynaklı 502 hataları önlenir.

6.5 Frontend

Hash'li statik varlıklar 1 yıl cache'lenir; `index.html` asla cache'lenmez (yeni deploy'da eski bundle'a takılma olmaz). Uzun listelerde sanal kaydırma (react-window) kullanılır.

7 Hata Yönetimi

7.1 Global exception handler

@ControllerAdvice ile merkezi: doğrulama hataları → 400 (+ alan hataları), erişim reddi → 403, bulunamadı → 404, beklenmeyen → 500 (+ loglama + temizlenmiş mesaj).

7.2 Frontend

- Axios interceptor: 401 → yeniden giriş.
- Global hata sınırı (error boundary).
- Her tabloda iskelet (skeleton) yükleme + boş durum (empty state).
- Kullanıcı bildirimleri toast ile.

7.3 Dış servis hataları

Her dış çağrı try/catch + log + güvenli fallback ile sarılır: fiyat sağlayıcı düşerse o tur atlanır; TCMB/EVDS düşerse son veri korunur; çeviri düşerse orijinal metin döner; LLM düşerse kural tabanlı yanıt; Kafka yoksa log sessizce düşürülür. Tek bir bozuk kayıt tüm toplu işlemi bozmaz.

8 Test ve Kod Kalitesi

8.1 Testler

- **Birim testleri (JUnit 5):** 50+ test sınıfı — servis katmanı (portföy, piyasa, enflasyon, risk, tahvil YTM, yenileme servisleri), controller (WebMvcTest), yardımcı sınıflar.
- **Entegrasyon:** H2 in-memory + Flyway göçleriyle repository testleri; `@SpringBootTest` context testi.
- **Kapsam:** JaCoCo; DTO/entity/config/dış istemci/scheduler sarmalları hariç; servis+controller+exception için hedef %80+.

8.2 SonarCloud Quality Gate — PASSED

Metrik (New Code)	Değer	Eşik	Durum
Security Rating	A	A	✓
Reliability Rating	A	A	✓
Maintainability Rating	A	A	✓
Coverage	%80.6	≥%80	✓
Duplicated Lines	%0.1	≤%3	✓
Security Hotspots Reviewed	%100	%100	✓

Tarama her `main` push'unda ve PR'larda CI içindeki `sonarcloud` job'ı ile tetiklenir; görünüm-katmanı boilerplate'i CPD kapsamından bilinçli hariç tutulmuştur (backend tam kapsamdır).

9 Dağıtım (Deployment) ve CI/CD

9.1 Docker Compose (geliştirme/demo)

Tek komutla (`docker compose up -d`) tüm yığın ayağa kalkar: Postgres, Keycloak, backend, frontend, gözlemlenebilirlik ve yardımcı servisler.

9.2 Kubernetes (Kustomize)

`k8s/` dizini base + overlay yapısındadır: `base` (Postgres StatefulSet, Keycloak Deployment + bootstrap Job, backend/frontend Deployment, Service'ler, Ingress), `overlays/dev` (kind/Docker Desktop), `overlays/prod` (HPA, anti-affinity, cert-manager TLS) ve `overlays/gke` (canlı hedef).

9.3 Google Kubernetes Engine (GKE)

- **GCE Ingress** (L7 HTTP(S) LB) + global statik IP; **Google-managed TLS** sertifikası, HTTP→HTTPS 301.
- **BackendConfig** ile servis bazlı sağlık kontrolü; **backend HPA** (CPU %70, 1–3 replica); **startupProbe** ile uzun açılış toleransı.
- **Frontend runtime-config**: Keycloak URL'i build yerine çalışma zamanında çözülür → tek imaj her ortamda çalışır.
- `SPRING_PROFILES_ACTIVE=prod,nokafka` ile çekirdek deploy'da Kafka log hattı kapatılır (broker yok).

9.4 CI/CD (GitHub Actions)

- **CI** (`ci.yml`): paralel job'lar — backend (Maven verify + JaCoCo), frontend (Vite build), docker-compose sözdizimi, SonarCloud taraması.
- **CD** (`cd.yml`): CI yeşil olunca GKE'ye deploy. Google Cloud'a **Workload Identity Federation** ile anahtarsız kimlik; imajlar commit SHA tag'iyle **Artifact Registry**'ye; `kubectl apply -k overlays/gke` + `rollout status` doğrulaması. Yalnızca dokümantasyon değişen push'larda deploy atlanır.

10 Veri Modeli

26 Flyway göçü (V1–V26). Başlıca tablolar:

Tablo	Açıklama
market_instruments	Sembol kataloğu (sağlayıcı sembolü + görünen ad, gecikme bayrağı)
market_quotes	Anlık fiyat (latest, change_abs/pct, previous_close, volume, as_of)
market_candles	Günlük OHLC mumlar — (instrument_id, day) tekil
portfolio_positions	Kullanıcı pozisyonları (symbol, quantity, avg_cost, purchase_date)
watchlist / watchlist_items	Takip listeleri
debt_instruments / debt_instrument_quotes	Tahvil/bono enstrümanları ve günlük getiri geçmişi
investment_funds	TEFAS fon kataloğu + getiri yüzdeleri
inflation_data_points	(period_date, country) tekil — endeks + aylık/yıllık değişim
deposit_rate_points	TCMB banka mevduat faizleri
exchange_rates	TCMB kurları (alış/satış/efektif, kaynak)
news_feeds / news_articles	RSS kaynak listesi + haberler (kaynak dili + çeviri cache kolonları)
viop_contracts	Vadeli işlem kontratları snapshot
viop_positions / viop_transactions	Simülasyon: long/short net pozisyon + işlem geçmişi
bond_positions / bond_transactions	Simülasyon: nominal/kupon/itfa pozisyonları + işlem geçmişi
price_alerts / notifications	Fiyat alarmları + uygulama içi bildirimler
user_preferences	Bildirim/dil/tema tercihleri

İndeksleme: her tabloda PK; bileşik tekil anahtarlar (mum, enflasyon, tahvil kotaları); sık sorgulanan kolonlarda azalan indeksler (haber published_at , kota as_of).

11 API Tasarımı

19 controller, tümü `/api/v1/...` önekiyle: Admin, Analysis, Bond, BondTrade, DepositRate, ExchangeRate, Inflation, InvestmentFund, Market, News, Notification, Portfolio, PriceAlert, TechnicalAnalysis, UserPreferences, UserProfile, Viop, ViopTrade, Watchlist.

- **OpenAPI/Swagger:** SpringDoc 2.7 — `/swagger-ui/index.html` , `/v3/api-docs` .
- **Doğrulama:** Bean Validation (`@NotNull` , `@DecimalMin` vb.); hatalar global handler'da standart formatta döner.

```
{
  "timestamp": "2026-06-05T13:00:00Z",
  "status": 400, "error": "Bad Request",
  "message": "Validation failed",
  "fieldErrors": [{"field": "quantity", "message": "must be positive"}]
}
```

12 Gözlemlenebilirlik (Observability)

12.1 Grafana dashboard'ları (4)

- **Backend Monitoring:** HTTP hız/hata, p95 gecikme, JVM heap, HikariCP havuzu, GC.
- **Business Metrics:** servis bazlı yenileme başarı/başarısızlık, süre, veri alım hızı (FX/enflasyon/haber/fiyat).
- **Prometheus Health** ve **OTel Collector Health**.

12.2 İş metrikleri

Her zamanlanmış görev için sayaç/zamanlayıcı: `bond_refresh_*`, `fx_refresh_*`, `inflation_refresh_*`, `news_refresh_*`, `price_refresh_*` (updated/failed/skipped), `price_alert_check_failure_total`.

12.3 Log analitiği (OpenSearch)

İndeks deseni `finans-logs-*`; seviye bazlı hacim, en aktif logger'lar, zaman serisi ve trace_id bazlı görselleştirmeler; kayıtlı aramalar (tümü / hata-uyarı).

13 Frontend Mimarisi

- **Yapı:** React 19 SPA; `pages/` (rota başına sayfa), `components/` (paylaşılan + özellik bazlı), `api/` (Axios istemcileri), `contexts/` (i18n, para birimi gösterimi), `hooks/`, `utils/`.
- **Yönlendirme:** React Router 7; korumalı rotalar Keycloak oturumuna bağlı.
- **Durum:** Zustand (hafif global), bağlamlar (i18n/para birimi/tema).
- **Çok dillilik:** Özel i18n sözlüğü (TR/EN); eksik anahtarlar TR'ye, sonra anahtarın kendisine güvenli düşer; 36 terimlik araç ipucu sözlüğü (TermInfo).
- **Tema:** CSS değişkenleriyle açık/koyu tema; `data-theme` ile anlık geçiş.
- **Grafikler:** recharts (pasta/bar), lightweight-charts (alan/getiri — canvas; CSS değişkeni renkler somut değere çözülür), klinecharts (detaylı mum + çizim araçları + indikatör senkronizasyonu, çoklu karşılaştırma).
- **Bildirim:** merkezi `notify` yardımcı (toast), kullanıcı tercihlerine duyarlı.
- **Modal altyapısı:** Escape/arka plan ile kapanma; işlem sırasında kilitlenebilir (yarım işlem iptalini önler).
- **Para birimi gösterimi:** Orijinal / TL / USD dönüşümü tüm tablolarda tutarlı.

14 Bilinen Sınırlamalar

- **Veri sağlayıcı limitleri:** Ücretsiz uçlarda IP bazlı kısıtlamalar; üretimde ücretli sağlayıcı (TwelveData/Finnhub) önerilir.
- **EVDS3 oturumu:** API anahtarı kullanılır; bazı uçlar oturum çerezi gerektirebilir (süresi dolunca yenileme).
- **Çeviri kalitesi:** LibreTranslate hızlı ama jargonda zayıf; üretimde DeepL/OpenAI alternatif.
- **In-memory cache:** Caffeine — çok örnekli dağıtımda tutarlılık için Redis'e geçiş hazır.
- **Gerçek zamanlı push yok:** Polling tabanlı; alarmlar toplu kontrol.

15 Geliştirici Onboarding

1. Önkoşullar: Docker, Docker Compose, Git, ~8 GB RAM.
2. `.env.example` → `.env` kopyalayın (anahtarlar/sırlar).
3. `docker compose up -d`

Servis	Adres
Frontend	<code>http://localhost</code>
Backend API / Swagger	<code>http://localhost:8080/swagger-ui/</code>
Keycloak admin	<code>http://localhost:8090/admin</code>
Grafana	<code>http://localhost:3001</code>
Jaeger	<code>http://localhost:16686</code>
OpenSearch Dashboards	<code>http://localhost:5601</code>
Mailpit	<code>http://localhost:8025</code>

Frontend dev: `cd frontend && npm install && npm run dev` · Backend dev: `cd backend && ./mvnw spring-boot:run` · Test: `./mvnw test`

— Belge sonu —